



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386

6. Software Engineering best practices

Version Control: git

What is this?

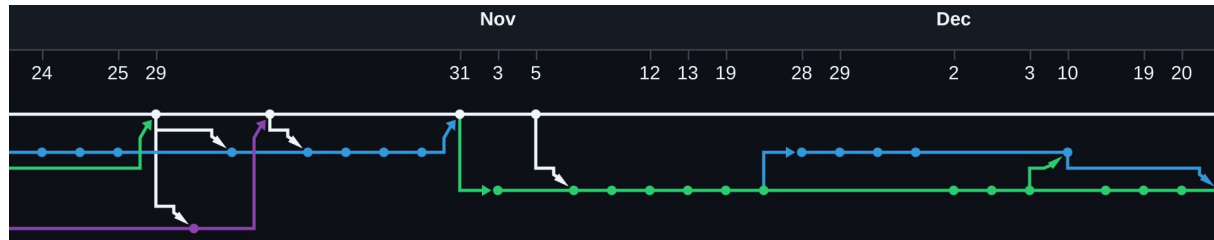
A tool to allow you to track and revert changes, and collaborate with others (change management).

Why is this important?

- Allow versioning of the code and continuing functionality.
- Allow simultaneous changes to the same files.
- Fundamental for reproducing historic states in the line of development.
- Development follows a story and allows other users to build confidence in your work.

Version Control in practice: git

- Create a repo on GitHub
- Clone the repo to your local machine
- Checkout a branch and make changes
- `git add`, `git commit` and `git push`: IDEs such as VSCode make it easy for you!
- Observe how your repo changes on GitHub
- GitHub offers great learning labs: GitHub skills <https://skills.github.com/>
- roadmap.sh offers a git roadmap: <https://roadmap.sh/git-github>



Development workflows: GitHub-flow

What is this?

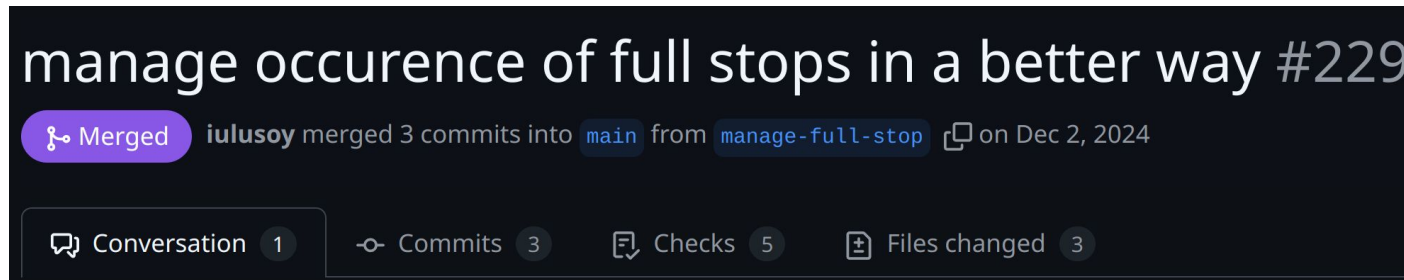
GitHub-flow is a lightweight workflow with creating branches, making changes, opening Pull Requests, running Continuous Integration, and requesting code review, together with Issues and Kanban project boards.

Why is this important?

- Manage how changes are incorporated in the software.
- Track progress in your project and highlight bottlenecks.
- Adhere to development guidelines, ensuring the implementation follows defined rules to ensure software quality.

Development workflows in practice: GitHub-flow

- After you made changes in your git branch, open a Pull Request on GitHub
- Observe how the PR highlights the changes in the line of development
- You can link issues, comment on the PR and run automated checks
- See GitHub skills <https://skills.github.com/>
- See roadmap.sh <https://roadmap.sh/git-github>



Requirements engineering and continuous delivery

What is this?

Requirements engineering is the process of translating stakeholder requirements on the research software into defined tasks. Early delivery and iteration over it allows refinement of the requirements and tasks.

Why is this important?

- Ensure that the software fulfills its purpose. In research software, requirements engineering is closely intertwined with the research process and subject to frequent changes.
- Understand the problem the software should solve and map this onto an efficient technically feasible solution considering all constraints.
- Allows prioritization of requirements/tasks and decision-making. Decisions should be documented together with the requirements.

Requirements engineering in practice

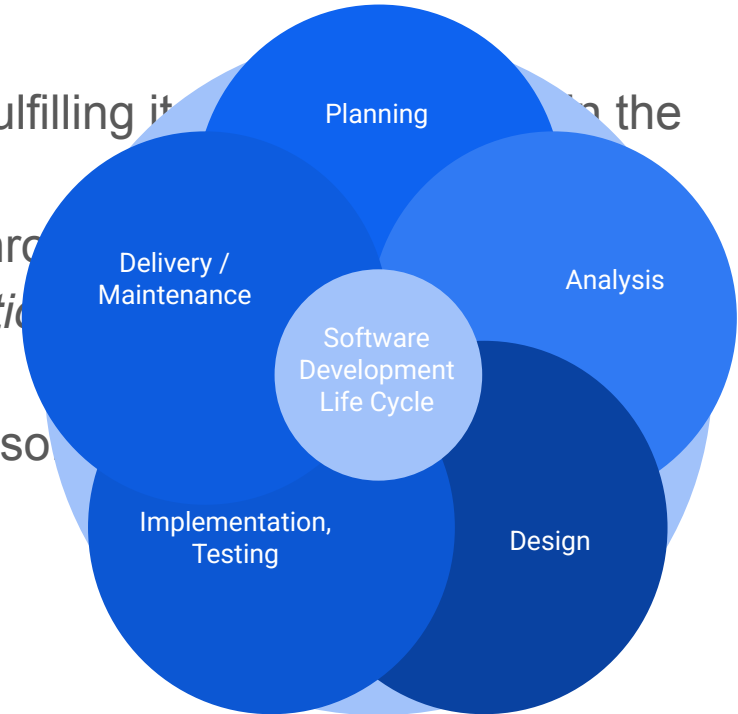
- Functional requirements (what the software should do: features)
- Non-functional requirements (how the software performs a task: ie performance, security)
- Domain requirements (specific to the domain: ie. Healthcare)
- Use tools such as draft.io or miro to gather requirements

As a <type of user>, I want
<some goal> so that <some
reason>.

As a <researcher>, I want
<to obtain the
probability of a class>
so that <I can classify
incoming images>.

Continuous delivery in practice

- Deliver early to find out if your software is fulfilling its purpose in the right direction
- Use quality control to allow early delivery through GitHub-flow (*keep your main branch operational*)
- Following agile / lean principles
- Use git to allow consistent usability of your software



Project management: Kanban boards

What is this?

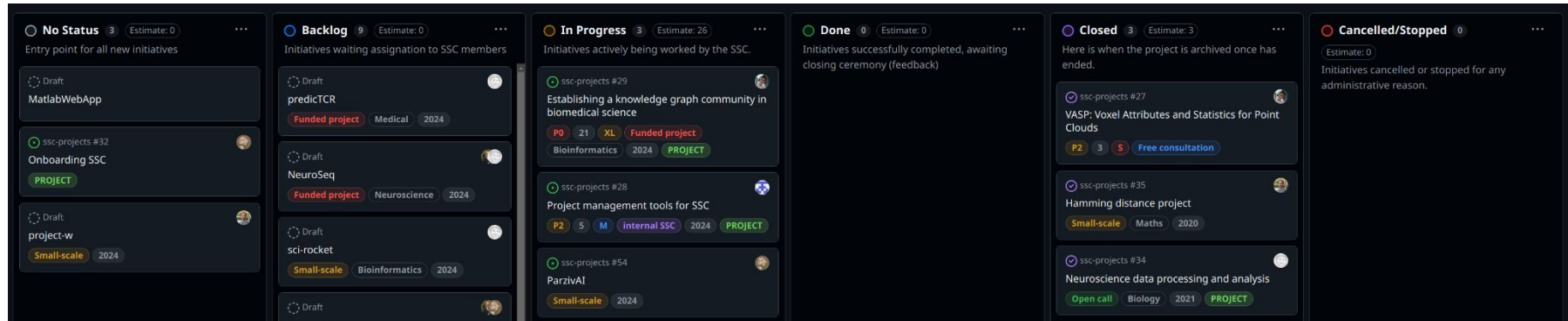
Project management is used to track progress, identify intertwined or dependent processes, and allows visual access to the project's status.

Why is this important?

- Ensure that the software development moves in the right direction.
- Increase the flow of ongoing work.
- Allow prioritization of tasks and understand interdependencies and bottlenecks in the development.

Project management in practice

- Use a Kanban board to organize tasks
- Separate tasks into backlog/todo, in progress, done
- Prioritize tasks and assign contributors/identify necessities
- For example, GitHub projects



Project planning: Architecture and design

What is this?

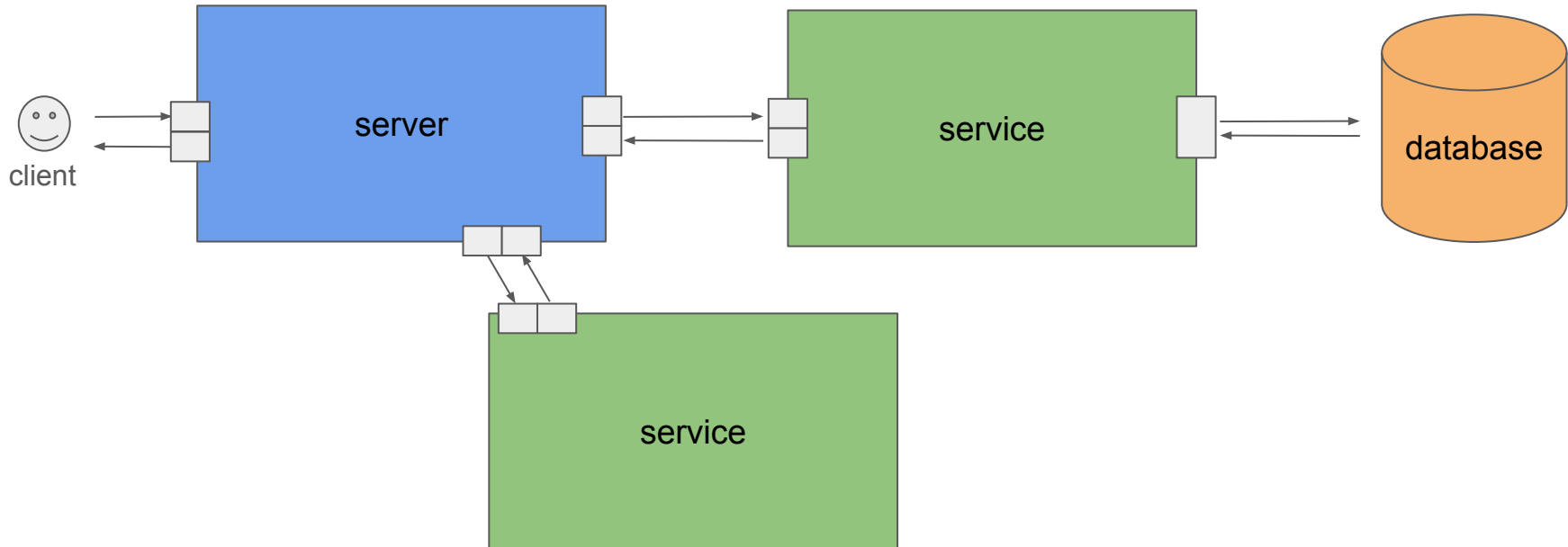
Software architecture describes how the system is composed of different pieces, and the interplay of the components. Design refers to the actual implementation of the requirements in the system as a whole and the different components.

Why is this important?

- Makes the software efficient and allow re-use of functionalities.
- Allows extensions and additions of features at a later stage without major refactoring.
- Makes the software maintainable.

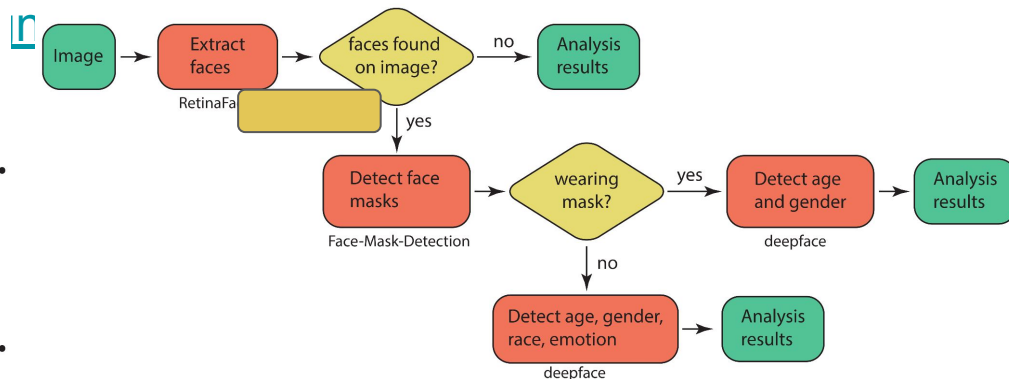
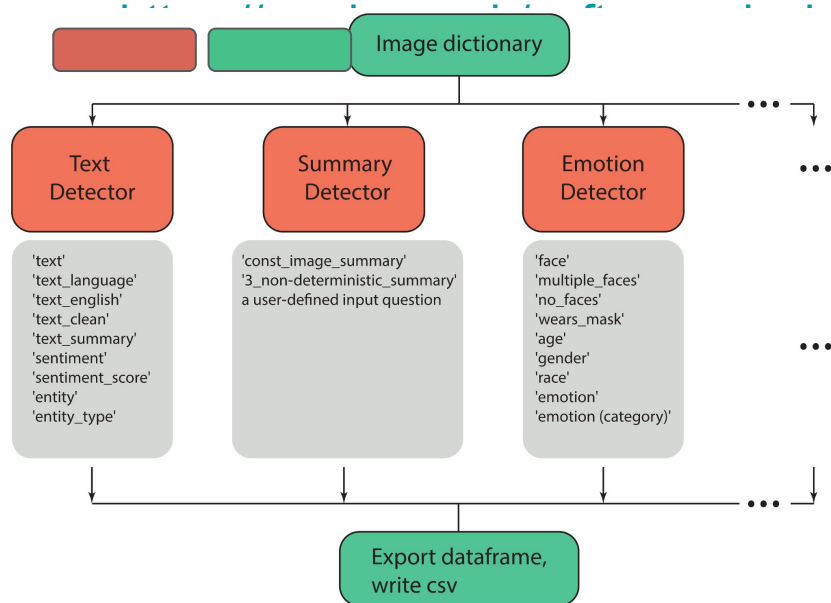
Architecture in practice

- Use (black/white) box diagrams to identify components and their interactions
- <https://roadmap.sh/software-design-architecture> / miro / draft.io / draw.io



Design in practice

- Map the input/output, data formats, transformations / logic / processes



```

image_summary_vqa_detector = ammicco.SummaryDetector(image_dict,
    analysis_type="summary_and_questions",
    model_type="base")

for key in image_dict.keys():
    image_dict[key] = image_summary_vqa_detector.analyse_image(
        subdict=image_dict[key], analysis_type="summary_and_questions",
        list_of_questions = list_of_questions)
  
```

Quality management: Testing and continuous integration

What is this?

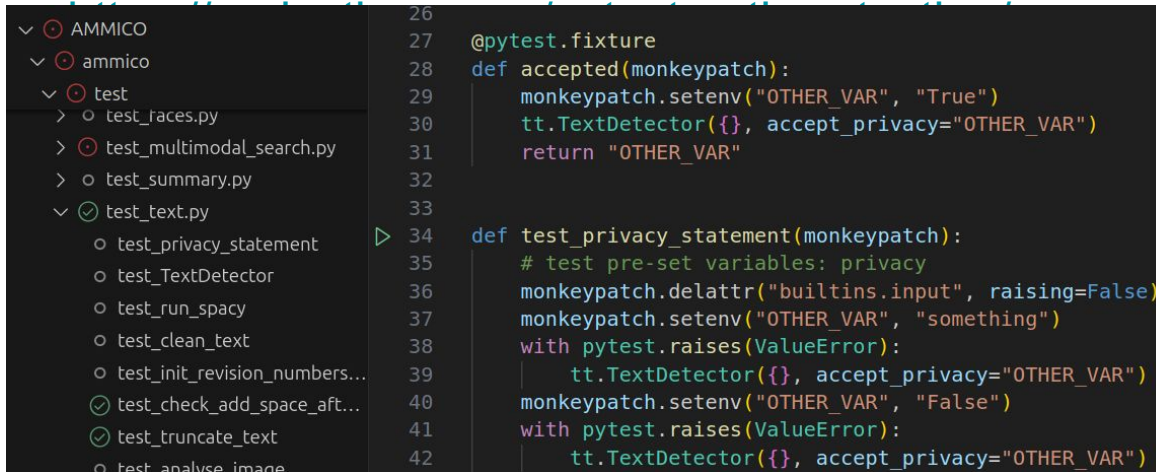
Quality management ensures that the software is producing correct results and that the results are reproducible.

Why is this important?

- As a scientist, you should take utmost care to conduct your science in accordance with the scientific code of conduct.
- Yourself or other researchers may need to reproduce results for further studies.
- Ensure that nothing breaks when you implement changes/new features.

Testing in practice

- Use testing frameworks such as pytest
- Write tests in a tests/ folder: unit tests, integration tests, system tests, compatibility tests, ...
- To learn how to use pytest: <https://docs.pytest.org/en/stable/>,



```

26
27 @pytest.fixture
28 def accepted(monkeypatch):
29     monkeypatch.setenv("OTHER_VAR", "True")
30     tt.TextDetector({}, accept_privacy="OTHER_VAR")
31     return "OTHER_VAR"
32
33
34 def test_privacy_statement(monkeypatch):
35     # test pre-set variables: privacy
36     monkeypatch.delattr("builtins.input", raising=False)
37     monkeypatch.setenv("OTHER_VAR", "something")
38     with pytest.raises(ValueError):
39         tt.TextDetector({}, accept_privacy="OTHER_VAR")
40     monkeypatch.setenv("OTHER_VAR", "False")
41     with pytest.raises(ValueError):
42         tt.TextDetector({}, accept_privacy="OTHER_VAR")

```

Testing of non-deterministic processes

Try to make processes deterministic

For example: Use specified random seed.

Separate deterministic and non-deterministic processes and test separately

For example: Input processing can be tested separately from model prediction.

Test for output parameters and properties that remain constant

For example: Number of predictions, feature length, etc.

Include multiple valid outputs in your tests

For example: Three most probable classifications.

Robustness: A robust model is more likely to behave like a deterministic system

Make sure your model output is stable under a range of conditions.

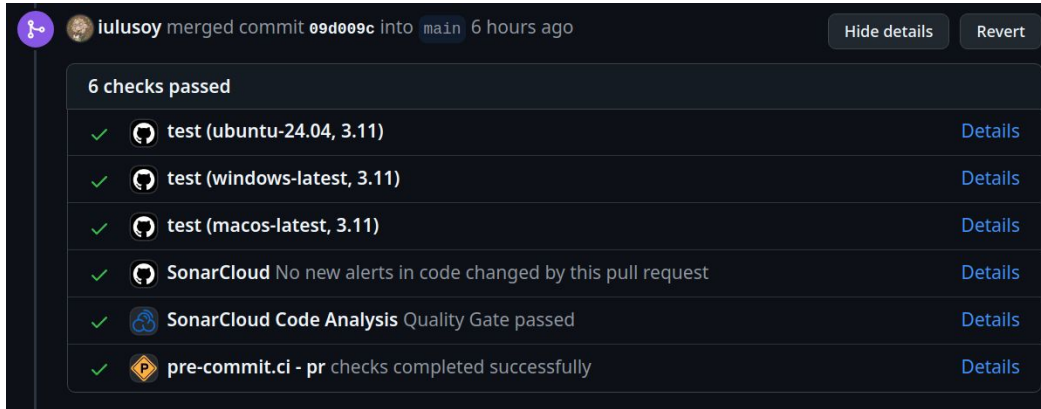
Accuracy: The model accuracy will affect the testing strategy

A higher accuracy leads to more consistent predictions.

Distribution: You can also test for the distribution of results rather than the results themselves

Continuous integration in practice

- Set up your tests to be automatically run by GitHub actions
- Include code linter and quality control in your actions
- These should be set up to run automatically when you open a Pull Request
- GH actions, codecov, sonarcloud, snyk, pre-commit, code formatting (black), GitHub Guardian, dependabot



The screenshot shows a GitHub pull request interface. At the top, it says "iulusoy merged commit 09d009c into main 6 hours ago". Below this, a section titled "6 checks passed" lists the following checks:

- ✓ test (ubuntu-24.04, 3.11) [Details](#)
- ✓ test (windows-latest, 3.11) [Details](#)
- ✓ test (macos-latest, 3.11) [Details](#)
- ✓ SonarCloud No new alerts in code changed by this pull request [Details](#)
- ✓ SonarCloud Code Analysis Quality Gate passed [Details](#)
- ✓ pre-commit.ci - pr checks completed successfully [Details](#)

Software Management Plans

What is this?

Software Management Plans (SMPs) help to identify goals and the means required to pursue the goals in practice.

Why is this important?

- Identify criticality and required maturity of your software.
- Identify which measures are needed to ensure compliance of your software with the intended goals.
- Quantify milestones and tools for the intended purpose.

SMPs in practice



SCIENTIFIC
SOFTWARE
CENTER



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386



RDMO for MPG

[Back to project](#)

Software Project Schedule

When does the software project start?

When does the software project end?

Software Project Management

Which software development process is defined? How will process roles be assigned?

How do you track the different tasks and use cases?

Will there be a specification document (briefly) outlining the most important requirements?

Software Development Requirements

Are there institutional requirements for software development?

Are there requirements regarding the software development from other parties?

Technical

Code

Which programming language(s) do you plan to use?

Which technology or process is used for versioning?

Third Party Components and Libraries

Which external software components will be used? What dependencies on software libraries do exist? How do you document this?

What licences are on the third-party software components?

What is the process to keep track of the external software components? Can critical dependencies be eliminated or mitigated?

Do you plan to use third party web services?

Does the software refer to other software projects or objects?

Infrastructure

What infrastructure resources are needed? To what extent?

- Use the SMPs provided by the Max Planck digital library
- Helps you with your requirements and project management
- <https://rdmo.mpd.l.mpg.de/>

Documentation

What is this?

Documentation can be comments, docstrings, readme's, tutorials, demonstration notebooks, and contains technical and domain-specific / application-specific descriptions of the software.

Why is this important?

- Document what the software can and cannot do, and parameter ranges.
- Allow others to install and use your software (or yourself, at a later time).
- Allow others to contribute to your software.

Documentation in practice



text module

```
class text.PostprocessText(mydict: dict | None = None, use_csv: bool = False, csv_path: str | None = None,
analyze_text: str = 'text_english')
```

Bases: `object`

analyse_topic(return_topics: int = 3) → tuple

Performs topic analysis using BERTopic.

Parameters: `return_topics` (int, optional) – Number of topics to return. Defaults to 3.

Returns: `tuple` – A tuple containing the topic model, topic dataframe, and most frequent topics.

get_text_df(analyze_text: str) → list

Extracts text from the provided dataframe.

Parameters: `analyze_text` (str) – Column name for the text field to analyze.

Returns: `list` – A list of text extracted from the dataframe.

get_text_dict(analyze_text: str) → list

Extracts text from the provided dictionary.

Parameters: `analyze_text` (str) – Key for the text field to analyze.

Returns: `list` – A list of text extracted from the dictionary.

- Use tools like sphinx and makedirs to render docstrings and markdown at minimal effort
- Include jupyter notebooks that showcase use of your software - these can be run on google colab
- Document dependencies in a requirements file and provide installation instructions

Deployment: Runtime environment / containerisation

What is this?

Deployment information such as runtime environments or containers allow easy adaption as they provide direct access to running the software without installation and dependency conflicts.

Why is this important?

- A big step towards reproducibility and transferability of your approach.
- The software ecosystem changes quickly, and this allows to preserve a snapshot that can be shared and run easily.

Containerisation in practice

- Use docker to provide build instructions for containers, and possibly deploy the containers on Dockerhub for anyone to download and use
- Docker roadmap <https://roadmap.sh/docker>, official tutorial <https://docs.docker.com/>

```
Dockerfile > FROM
1 FROM jupyter/base-notebook
2
3 # Install system dependencies for computer vision packages
4 USER root
5 RUN apt update && apt install -y build-essential libgl1 libglu1-mesa-dev libsm6 libxrender1 libxext6
6 USER $NB_USER
7
8 # Copy the repository into the container
9 COPY --chown=${NB_UID} . /opt/ammico
10
11 # Install the Python package
12 RUN python -m pip install /opt/ammico
13
14 # Make JupyterLab the default for this application
15 ENV JUPYTER_ENABLE_LAB=yes
16
17 # Export where the data is located
18 ENV XDG_DATA_HOME=/opt/ammico/data
```

Software Licensing

What is this?

A software license states the terms of use, re-use and distribution, among others, without violating copyrights, and defines responsibilities.

Why is this important?

- So that others may use your code, and to prevent misuse.
- So that others may contribute to your code.
- So that the responsibilities for how the software is used are clear.
- Establishes the rights of all parties involved with the software.

Software licensing in practice

- Use the provided templates from GitHub: Either at repository creation or when adding a new file called LICENSE
- Permissive open-source license: BSD 2-Clause, MIT, Apache License 2.0
- Copyleft open-source license: GNU version 3, LGPL
- Proprietary licenses: Do not only keep your project close-source and potentially less visible, but also carry responsibilities for contract fulfillment
- <https://opensource.org/licenses>, <https://choosealicense.com/>
- **When using/incorporating third-party software:** ie. use of open-source libraries - is the third-party code distributed with your software? If so, compatibility needs to be confirmed!

Tooling and generative AI

Tooling: Integrated Development Environment (IDE)

What is this?

An IDE is a software application that combines tools for software development, such as a code editor, linter, compiler, debugger, and automation utilities.

Why is this important?

- Provides you with all required tools in one place, making software development more efficient.
- Helps you keep your code clean.
- Many different IDE's available:

<https://github.com/zeelsheladiya/Awesome-IDEs>

IDEs in practice

- For example: VSCode <https://code.visualstudio.com/>

Visual Studio Code documentation

Get Familiar with Visual Studio Code and learn how to code faster with AI.

Getting started



Download Visual Studio Code

Download VS Code for Windows, macOS, or Linux.



Getting started

Discover the key features of VS Code with the step-by-step tutorial.



Code faster with AI

Get started with GitHub Copilot, your AI coding assistant.

Code with rich features



Code in any language

Write code in your favorite programming language.



Version control

Built-in support for git and many other source control providers.



Debugging

Debug your code without leaving your editor.



Testing

Run automated tests and view test coverage to validate your code.



Code anywhere

Use a container, remote machine, or WSL as your development environment.



Videos

Watch the introduction videos to learn more.

Top Extensions

Enable additional languages, themes, debuggers, commands, and more. VS Code's growing community shares their secret sauce to improve your workflow.



Python
ms-python
🔖 182.3M



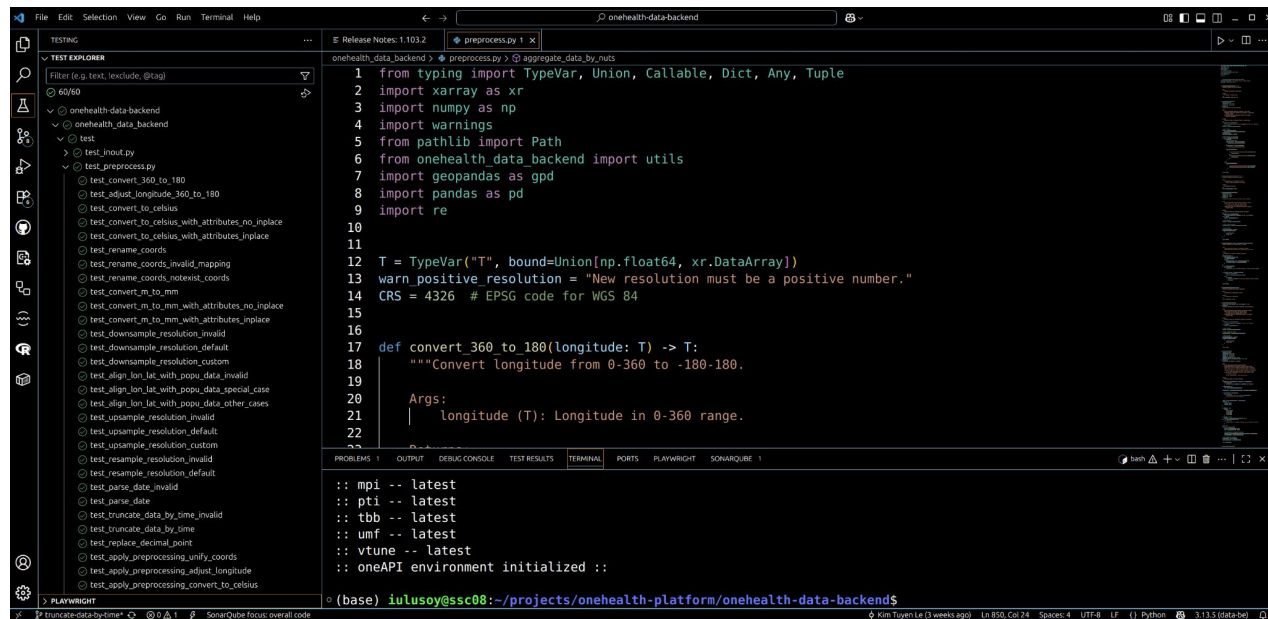
C/C++
ms-vscode
🔖 86.9M



GitHub Copilot
GitHub
🔖 49.6M



Extension Pack for Java
vscode
🔖 38.0M

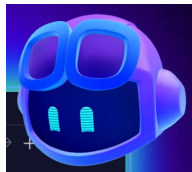


AI-supported software development

What is this?

The software development process can be supported by AI in many ways:

- Code completion
- Code generation
- Code analysis
- Code quality
- Documentation
- Debugging
- Security enhancement
- Architecture/UX design
- Full-cycle development (AI agents)



Why is this important?

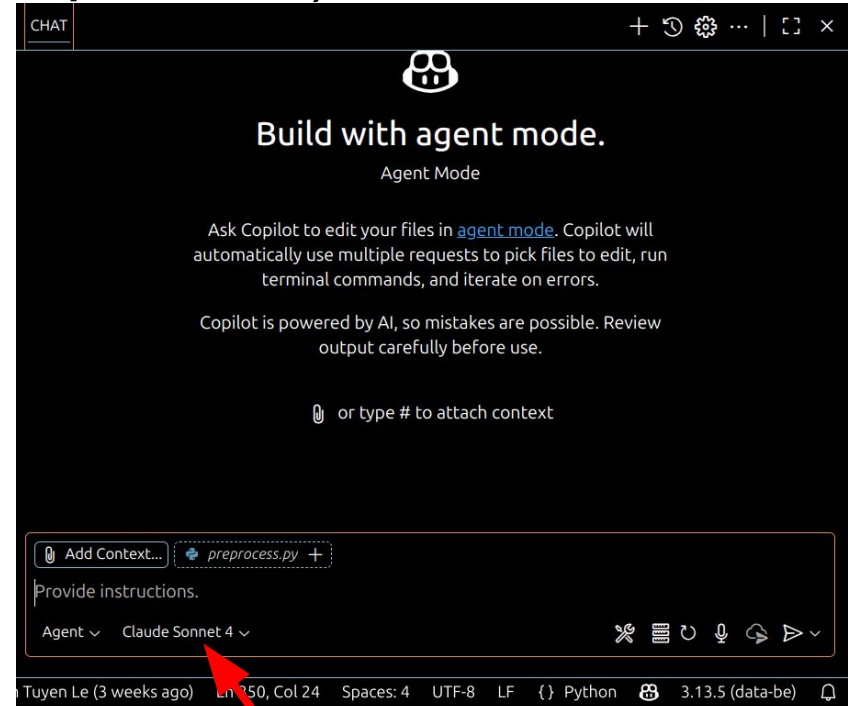
- AI has become an integral part of software development and can help you write better code.
- You can improve your skills by interacting with AI systems as you learn to understand why you should write something like this, or also why not.

AI-supported software development in practice

- For example: GitHub Copilot <https://github.com/features/copilot> - get free access as a student <https://github.com/education> - many other tools available as well, for example
 - Code analysis: ie. SonarQube Cloud integration to GitHub, snyk
 - Security enhancement: ie. snyk
 - UX design: ie. Figma
 - A list of tools: <https://github.com/jamesmurdza/awesome-ai-devtools>, <https://github.com/ikajjua/Awesome-AITools>
- Copilot is integrated into VSCode: Code completion, code generation, code summarization, generation of docstrings and documentation content
- Chat mode or agentic mode
- Also allows to chats on GitHub and can carry out Pull Requests by itself through agentic mode

GitHub Copilot: Models (as of Sept. 2025)

Anthropic Claude 3.5 Sonnet in Copilot	Enabled ▾
You can use the latest Claude 3.5 Sonnet model. Learn more about how GitHub Copilot serves Claude 3.5 Sonnet.	
Anthropic Claude 3.7 Sonnet in Copilot	Enabled ▾
You can use the latest Anthropic Claude 3.7 Sonnet model.	
Anthropic Claude Sonnet 4 in Copilot	Enabled ▾
You can use the latest Anthropic Claude Sonnet 4 model. Learn more about how GitHub Copilot serves Anthropic Claude Sonnet 4.	
Google Gemini 2.0 Flash in Copilot	Enabled ▾
You can use Google's Gemini 2.0 Flash model in Copilot. Learn more about the public preview of Gemini 2.0 Flash.	
Google Gemini 2.5 Pro in Copilot	Enabled ▾
You can use the latest Google Gemini 2.5 Pro model. Learn more about how GitHub Copilot serves Google Gemini 2.5 Pro.	
OpenAI o4-mini in Copilot Preview	Enabled ▾
You can use the latest OpenAI o4-mini model. Learn more about how GitHub Copilot serves OpenAI o4-mini.	
OpenAI GPT-5 in Copilot Preview	Enabled ▾
You can use the latest OpenAI GPT-5 model. Learn more about how GitHub Copilot serves OpenAI GPT-5.	
OpenAI GPT-5 mini in Copilot Preview	Enabled ▾
You can use the latest OpenAI GPT-5 mini model. Learn more about how GitHub Copilot serves OpenAI GPT-5 mini.	
xAI Grok Code Fast 1 in Copilot Preview	Enabled ▾
If enabled, you can access and send data to xAI Grok Code Fast 1. Learn more.	



select model

AI-supported software development: User roles

Chat user, ie. ChatGPT	Copilot user, ie. code completion and code generation	Chat-and-Copilot user, ie. code completion and code generation iterates with chat	Agentic user up to vibe coder
- Get help and learn through explanations and summarization	- Increase efficiency through passing over cumbersome tasks such as boilerplate code, docstring generation or straightforward pieces of code	- Increase efficiency and learn through interacting more closely with the LLM, refactoring suggestions in an iterative manner	- Pass of responsibility for pieces of code, or complete parts of the code to the AI agent without following the AI implementation in detail

AI-supported software development: caveats

- Not understanding your code
- Creating code that is harder to maintain
- Mixing of code for different releases of the same library (ie. langchain 0.2.x, 0.3.x, especially in frontend development)
- Rewriting code that is actually provided by a library through a simple keyword
- Subtle hallucinations that introduce hard-to-spot mistakes
- AI only as good as the context
- Does not generate code in the same style as existing codebase (lack of context-awareness and preservation)
- And of course, copyright, legal and ethical considerations

Starting a new project

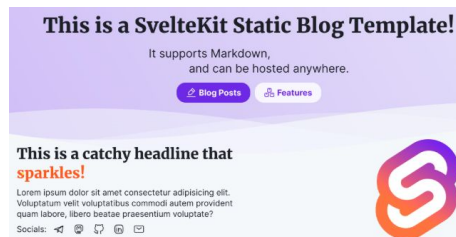
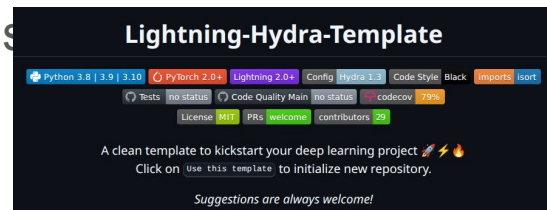
Project templates

What is this?

Project templates exist for example on GitHub. They include configurations for specific types of projects.

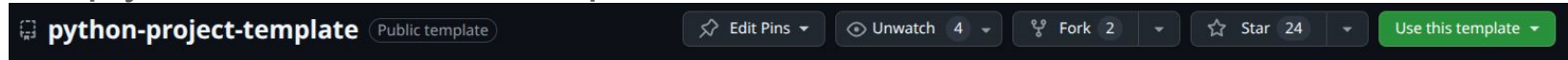
Why is this important?

- Project templates help you save time by providing complete configurations for testing, CI, and documentation, for example.
- Templates help you adhere to common practices, configurations, or

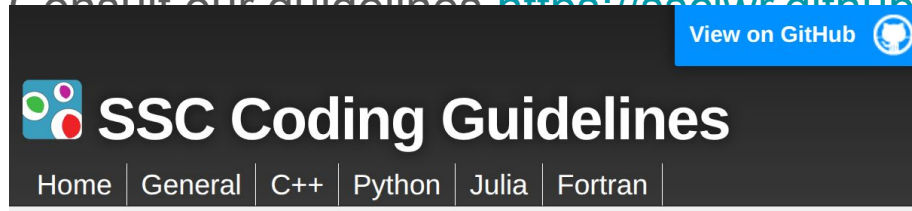


Project templates in practice

- For example: GitHub template repositories
- Simply click on “use this template”



- More configurable templates: cookiecutters (also available for data science)
- The SSC offers such template repositories for
 - C++
 - Python
 - Fortran
 - Matlab
- Consult our guidelines <https://ossiv.github.io/guidelines/>



Summary: Software Engineering best practices

Best practices: Building blocks for your software project

